

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BLOCKCHAIN AND APPLICATIONS
CAPSTONE PROJECT REPORT
TOPIC: NFT MARKETPLACE WITH FRACTIONAL ROYALTIES

Course Instructor: Mr. Nguyễn Bình Minh

Group: 3

Members:

Chu Văn An – 20235581

Lê Duy Anh – 20225565

Trần Chí Hiếu – 20225572

Class ID: 168076

Hà Nội, tháng 6 năm 2026

Table of Contents

1. Introduction and Task Assignment
2. Technical Requirement 1: Advanced ERC-721 with EIP-2981 Multi-Receiver Royalties
3. Technical Requirement 2: Pull-Payment Splitter (Claim-Based Withdrawal)
4. Technical Requirement 3: Atomic Swaps in buyNFT()
5. Security Analysis
6. How the Components Work Together
7. Test Scenarios
8. Conclusion

1. Introduction and Task Assignment

1.1 Introduction

The NFT Fractional Marketplace is a blockchain-based platform that allows users to mint, list, buy, and sell NFTs while automatically splitting royalty fees among multiple stakeholders. Unlike standard NFT marketplaces that send all royalty fees to a single recipient, this project distributes fees proportionally among multiple royalty receivers defined at minting time.

The system is composed of two smart contracts written in Solidity:

- FractionalRoyaltyNFT.sol — An ERC-721 NFT contract extended with EIP-2981 royalty support and a custom multi-receiver royalty array.
- NFTMarketplace.sol — A marketplace contract that handles listing, buying, and automatic royalty distribution using a pull-payment (claim-based) pattern.

This report focuses on three core technical requirements: (1) extending ERC-721 with EIP-2981 multi-receiver royalties, (2) implementing a secure pull-payment splitter, and (3) ensuring atomic swaps where NFT transfer and fund distribution occur in a single transaction.

1.2 Task Assignment

Chu Văn An – 20235581: Group Leader, write and test the main branch, do the slides.

Lê Duy Anh – 20225565: Write and test the backup (Remix IDE) version, do the demo and presentation.

Trần Chí Hiếu – 20225572: Create a Remix IDE version of the main branch, write the report.

2. Advanced ERC-721 with EIP-2981 Multi-Receiver Royalties

2.1 What is EIP-2981?

EIP-2981 (ERC-2981) is an Ethereum standard that defines a way for NFTs to communicate royalty information to marketplaces. It introduces a single function:

```
royaltyInfo(uint256 tokenId, uint256 salePrice)
```

This function returns (address receiver, uint256 royaltyAmount), telling the marketplace who to pay and how much. However, the standard only supports a single receiver. This project extends that concept using a custom multi-receiver array.

2.2 Multi-Receiver Implementation

The FractionalRoyaltyNFT contract stores an array of RoyaltyReceiver structs per token:

```
struct RoyaltyReceiver {  
    address receiver;  
    uint96 share; // in basis points (e.g., 5000 = 50%)  
}
```

```
mapping(uint256 => RoyaltyReceiver[]) private _tokenRoyalties;
```

When mintWithRoyalties() is called, the contract validates that all shares add up to exactly 10000 (100%), ensuring no over- or under-payment. Each receiver address is stored in the array for that specific token ID.

2.3 How royaltyInfo() Works

The royaltyInfo() function from EIP-2981 returns this contract itself as the receiver and a flat 10% royalty rate. The marketplace then calls getRoyaltyReceivers() to retrieve the individual receivers and their shares,

distributing the 10% royalty among them proportionally. This is the “bridge” between the standard EIP-2981 interface and the custom multi-receiver logic.

3. Pull-Payment Splitter (Claim-Based Withdrawal)

3.1 The Problem

In a typical NFT sale, the marketplace must distribute funds to the seller and potentially multiple royalty receivers within the same transaction. If the marketplace pushes ETH directly to many addresses via `.call{value: ...}()`, several issues arise:

- **Reentrancy Attack:** A malicious receiver contract could call back into the marketplace before the transaction ends, potentially draining funds.
- **Out of Gas (OOG):** If there are many royalty receivers, the transaction may run out of gas trying to send ETH to each one individually, causing the entire sale to fail.
- **Failed Transfers:** If any single receiver's contract rejects the ETH (e.g., a contract without a `receive()` or `fallback()`), the entire purchase transaction reverts.

3.2 The Solution: Pull-Payment Pattern

Instead of "pushing" ETH to recipients during the sale, the marketplace uses a "pull-payment" (claim-based) pattern. Funds are tracked in a mapping:

```
mapping(address => uint256) public accruedBalances;
```

When a sale happens in `buyNFT()`, the contract does NOT send ETH to anyone directly. Instead, it increments the `accruedBalances` mapping for the seller and each royalty receiver. Users must call `withdrawPayments()` separately to claim their funds.

3.3 `withdrawPayments()` Implementation

```
function withdrawPayments() external nonReentrant {  
    uint256 amount = accruedBalances[msg.sender];  
    require(amount > 0, "No funds available for withdrawal");  
    accruedBalances[msg.sender] = 0; // Check-Effects-Interactions  
    (bool success, ) = payable(msg.sender).call{value: amount}("");
```

```
    require(success, "Transfer failed");  
}
```

This function follows the Checks–Effects–Interactions pattern:

- Check: Verifies the caller has a balance greater than zero.
- Effects: Sets the balance to ZERO before making the external call. This prevents reentrancy because even if the caller's `receive()` function tries to call `withdrawPayments()` again, their balance is already zero.
- Interaction: Sends ETH via `.call{value: amount}()`.

The `nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` provides an additional layer of protection at the function level.

4. Atomic Swaps in buyNFT()

4.1 What is Atomicity?

In blockchain, an “atomic” operation means that everything happens in one transaction — either all steps succeed, or the entire transaction reverts and no state changes occur. This is crucial for a marketplace because you never want a situation where:

- The buyer’s ETH is taken but the NFT is not transferred.
- The NFT is transferred but the seller does not receive payment.
- Royalty receivers are paid but the NFT stays with the seller.

4.2 The buyNFT() Transaction Flow

The buyNFT() function performs all operations within a single function call, ensuring atomicity:

```
function buyNFT(address nftAddress, uint256 tokenId) external payable
nonReentrant {
    // 1. Validate listing
    Listing storage listing = listings[nftAddress][tokenId];
    require(listing.isActive, "Not listed");
    require(msg.value == listing.price, "Incorrect payment");

    // 2. Deactivate listing (state change before external calls)
    listing.isActive = false;

    // 3. Calculate and distribute royalties (internal bookkeeping)
    uint256 totalRoyaltyAmount = 0;
    IFractionalNFT.RoyaltyReceiver[] memory receivers =
        IFractionalNFT(nftAddress).getRoyaltyReceivers(tokenId);
    for (uint256 i = 0; i < receivers.length; i++) {
        uint256 receiverShare = (royaltyAmount * receivers[i].share) / 10000;
        accruedBalances[receivers[i].receiver] += receiverShare;
```

```

}

// 4. Credit seller's proceeds
uint256 sellerProceeds = salePrice - totalRoyaltyAmount;
accruedBalances[seller] += sellerProceeds;

// 5. Transfer NFT (last external call)
IERC721(nftAddress).safeTransferFrom(seller, msg.sender, tokenId);
}

```

4.3 Why This Is Atomic

All steps are inside one transaction. Solidity reverts the entire transaction if any step fails. For example:

- If `safeTransferFrom()` fails (e.g., the NFT was already moved), the entire function reverts — the buyer gets their ETH back (since `msg.value` remains in the contract state as it was), and the `accruedBalances` updates are rolled back.
- If `msg.value` is less than `listing.price`, the function reverts before any state changes occur.
- The listing is marked inactive BEFORE external calls and fund distribution, preventing double-selling.

This pattern ensures that ETH only moves when the NFT moves, and royalties are only credited when the sale succeeds.

5. Security Analysis

5.1 Reentrancy Protection

The contract uses multiple layers of protection:

- Check-Effects-Interactions pattern in `withdrawPayments()` — balance set to zero before ETH is sent.
- OpenZeppelin's `ReentrancyGuard` (`nonReentrant` modifier) on `buyNFT()` and `withdrawPayments()`.
- Pull-payment pattern ensures no external calls are made to royalty receivers during the sale.

5.2 Denial of Service (Gas) Protection

By using the pull-payment pattern, the transaction cost of `buyNFT()` is bounded. Even if an NFT has 100 royalty receivers, the contract only updates a storage mapping (SSTORE operation) for each — it does not send ETH. This prevents “Out of Gas” attacks where a malicious minter could add many receivers to exhaust gas.

5.3 Input Validation

The contract validates critical inputs:

- `mintWithRoyalties()`: Total shares must equal exactly 10000. Receiver addresses must not be zero.
- `listNFT()`: Price must be greater than zero. Only the NFT owner can list. Marketplace must be approved.
- `buyNFT()`: Listing must be active. `msg.value` must match the listing price exactly.
- `withdrawPayments()`: Balance must be greater than zero.

6. How the Components Work Together

The complete sale flow demonstrates how all three technical requirements combine:

Step 1: Minting: The contract owner calls `mintWithRoyalties()` to create an NFT with an array of royalty receivers (e.g., Artist 70%, Manager 30%). The contract validates shares add to 100%.

Step 2: Approval: The NFT owner calls `setApprovalForAll(marketplace, true)` to allow the marketplace to transfer the NFT.

Step 3: Listing: The seller calls `listNFT()` with the NFT address, token ID, and price. The marketplace verifies ownership and approval.

Step 4: Purchase: The buyer calls `buyNFT()` with the correct ETH payment.

Inside this single atomic transaction:

- a. The listing is deactivated (prevents double-sell)
- b. Royalty information is read from `FractionalRoyaltyNFT`
- c. Royalty amounts are calculated per receiver and added to `accruedBalances`
- d. The seller's proceeds are added to `accruedBalances`
- e. The NFT is transferred to the buyer via `safeTransferFrom()`

Step 5: Withdrawal: Both the seller and royalty receivers call `withdrawPayments()` separately to claim their ETH. This uses the Check-Effects-Interactions pattern for security.

This design ensures that: (1) royalties are always distributed according to the fractions set at minting, (2) no single buyer or seller transaction can DoS the system, and (3) every purchase is atomic — all parties are credited or no one is.

7. Test Scenarios

The following scenarios can be used to validate the system:

7.1 Basic Mint and Royalty Verification

Mint an NFT with two royalty receivers (e.g., 5000 + 5000). Verify that `getRoyaltyReceivers()` returns the correct addresses and shares, and that `royaltyInfo()` returns the expected 10% fee.

7.2 Full Buy–Sell Flow

Mint → Approve → List → Buy — verify that the NFT changes owner, the seller's `accruedBalance` reflects the sale minus royalties, and each royalty receiver's balance is correct.

7.3 Reentrancy Attempt

Deploy a malicious contract that calls `withdrawPayments()` inside its `receive()` function. Verify the function only pays once due to the Checks–Effects–Interactions pattern.

7.4 Invalid Share Rejection

Attempt to mint with receivers summing to 9000 (not 10000). Verify the transaction reverts.

7.5 Double–Purchase Prevention

After an NFT is bought, verify that calling `buyNFT()` again for the same listing reverts.

8. Conclusion

The NFT Fractional Marketplace demonstrates three important smart contract design patterns that go beyond basic NFT trading:

- **Advanced ERC-721 with EIP-2981:** By storing an array of royalty receivers per token and using the standard `royaltyInfo()` function as a bridge, the contract achieves backward compatibility with standard marketplaces while supporting complex multi-receiver royalty splitting.
- **Pull-Payment Splitter:** The claim-based withdrawal pattern (`accruedBalances` mapping) eliminates reentrancy vulnerabilities and out-of-gas issues that plague push-based distribution systems.
- **Atomic Swaps:** The `buyNFT()` function ensures that NFT transfer and fund allocation happen in a single atomic transaction — either all state changes succeed or none do.

The combination of these patterns creates a robust, gas-efficient, and secure marketplace suitable for real-world use cases where NFT revenue needs to be shared among multiple stakeholders such as artists, publishers, platforms, and investors.